# DLDCA Outlab: Profiling and Optimisation

Weeks 7 and 8

November 6, 2026

**Read this first: where to run this lab**

`perf` needs the **hardware counters of a real CPU**. Use a **Software Lab machine over ssh**, or a **native Linux machine**. **Do not use a virtual machine or WSL**: there `perf stat` prints `<not supported>` for `cycles` and the lab cannot be done.

```
$ scp -r 08_outlab <cse-username>@sl<N>-<MM>.cse.iitb.ac.in:~/
$ ssh <cse-username>@sl<N>-<MM>.cse.iitb.ac.in
```

## Introduction

In Labs 7 and 8 you were told where the problem in a program was. **In this outlab you find it yourself**, in programs you did not write: you measure, find the slow part, name the hardware effect behind it (**cache misses, mispredicted branches, no SIMD**), fix it, and compare the numbers before and after.

You already know `perf stat`: it tells you **how many** cycles, misses and branches a program used. The new tool is `perf record` with `perf report`: it tells you **where**.

- **Task 1**: measure two programs with `perf`. No code.
- **Task 2**: two small programs, one problem each (branches, memory order).
- **Task 3**: rewrite loops with SSE2.
- **Task 4**: a larger program with several problems.
- **Task 5**: a program whose hotspot moves when the data grows.
- **Task 6**: optional and hard.

**For the lab exam, practise Tasks 1 to 3.** Tasks 4 and 5 use the same ideas on bigger programs, so do not worry if you do not find every improvement. Nothing in this outlab is marked on speed.

## Necessary tools

`g++`, `perf`, `python3` and `gzip`. `valgrind` (installed on the lab machines) is needed for one check in Task 4. Check:

```
$ g++ --version
$ perf --version
$ valgrind --version
$ make sysinfo          # your CPU, caches, SIMD support, compiler
```

## Background: profiling with `perf`

`perf stat`, events, IPC, `-r` and `taskset` are in the **Week 7 slides**. `perf record`, `perf report` and `perf annotate` are in the **updated Week 8 slides** (Part 5), at the same link as the Week 8 slides and in this folder as `Week8-Performance-updated.pdf`. Have them open while you work. In short:

- `perf stat` **counts**: totals for the whole run.
- `perf record` **samples**: about 4000 times a second it notes which instruction is running. A function with 40% of the samples ran for about 40% of the time.
- `perf report --stdio --no-children` lists functions by their **self** share (their own code). **Start from the largest.**
- `perf annotate --stdio -l -M intel '<function>'` shows the share per instruction and per source line.
- `-e <event> -c <n>` samples every $n$ events of that kind (for example cache misses) instead of time.

**Note:** on AMD processors a sample lands a little after the instruction that caused it. Treat a sampled location as a hint and **confirm it with a change and** `perf stat`.

## Building and running

```
$ make <program> args="..."      # build and run on core 1
$ make <program>.out             # only build (then run perf on ./<program>.out)
$ make taskN-check               # correctness checks (tasks 2 to 5)
$ make clean
```

Tasks 1 to 3 are built with `-O0` (the machine code follows your C++ line by line). Tasks 4 and 5 use `-O2`, like real programs. Every build adds `-g`.

**Reference values** come from the machine in `reference_machine.txt`. **Yours will differ**: compare the **change** (before against after) on your own machine, not absolute times.

## How to work on Tasks 2 to 5

Write notes in `findings.txt` **before** you change any code, and in `changes.txt` as you go.

1. **Run it** and read the output.
2. **Count.** `perf stat` the given program. Compute the **IPC**, **branch miss rate** and **L1 miss rate**. Which number looks wrong?
3. **Find the hotspots.** `perf record -g`, then `perf report`. Note the top functions.
4. **Go one level deeper** on the top one: read its code, or `perf annotate` it.

5. **Diagnose**: one cause per hotspot, and the number that shows it.
6. **Change one thing**, the biggest hotspot first. Check the result, then `perf stat` again. Note it in `changes.txt`, also if it did not help.
7. **Repeat** until the profile is flat or the next change is not worth it.

### The causes you know

White rows are from Labs 7 and 8. **Gray rows are new in this outlab.**

| **Cause** | **What `perf` shows** | **General fix** |
|---|---|---|
| stride / cache misses (Lab 8 Task 2) | high `L1-dcache-load-misses`, low IPC | visit memory in the order it is stored; reuse data while it is in the cache |
| dependency chain (Lab 8 Task 1) | low IPC, misses on dependent loads | break the chain; do independent work in between; compute results once |
| branch misprediction (Lab 8 Task 3) | high `branch-misses` | turn the condition into arithmetic; make the branch predictable |
| not SIMD (Lab 8 Task 4) | many instructions, high IPC, no misses | process several elements per instruction |
| function call overhead | small functions with a large self share | let the compiler inline them; do more work per call |
| allocation | `sys` time, many `page-faults`, kernel symbols | allocate once and reuse; free what you allocate |
| wasteful work | instruction count far above the work needed; a simple loop dominates | search or compute once, then look it up |
| data too big for the cache | hotspot and misses grow with input size | touch the data in a cache-friendly order; shrink what you store |

## Submission

Edit only the files marked `(TODO)`. If a check keeps failing for no clear reason, run `make clean` and try again.

```
your-roll-no/
|- task1/
    |- answers.txt (TODO)
|- task2/
    |- repeated_array.cpp (TODO)
    |- weird_matrix.cpp (TODO)
    |- findings.txt (TODO)
    |- changes.txt (TODO)
|- task3/
    |- mask.cpp (TODO)
    |- optim.cpp (TODO)
    |- findings.txt (TODO)
    |- changes.txt (TODO)
|- task4/
    |- grid_bfs.cpp (TODO)
    |- expected_output.txt
    |- findings.txt (TODO)
    |- changes.txt (TODO)
|- task5/
    |- packets.cpp (TODO)
    |- expected_output.txt
    |- findings.txt (TODO)
    |- changes.txt (TODO)
|- task6/   (optional)
    |- spawn_sim.cpp (TODO, optional)
```

`./build_archive.sh` packs these files into `your-roll-no.tar.gz` (roll number in lowercase).

# Tasks

# Task 1: Measuring

Before making a program faster, **measure it**. The counters tell you whether the core is **busy or waiting**, and **what** it is waiting for. You write no code in this task. Fill `task1/answers.txt (TODO)` as you go.

### Part A: gzip

`gzip` is on every Linux machine. You cannot read its code, so **the counters are all you have**. Make a 79 MB text file, then compress it and throw the output away:

```
$ make task1-input
$ taskset -c 1 perf stat \
    -e task-clock,cycles,instructions,branches,branch-misses \
    gzip -c task1/input.txt > /dev/null
```

The output looks like this (your numbers will differ):

```
Performance counter stats for 'gzip -c task1/input.txt':

         1,586.23 msec task-clock
     7,045,253,025      cycles
    16,821,604,786      instructions
     3,773,255,901      branches
         5,005,048      branch-misses

       1.588870536 seconds time elapsed
```

A1. Compute the **IPC** (`instructions / cycles`) and the **branch miss rate** (`branch-misses / branches`). Is the core **busy or waiting**?

A2. Add `-r 5` after `perf stat`. It runs the program 5 times and prints the mean with the spread (`+-`). Does the time change from run to run?

```
Performance counter stats for 'gzip -c task1/input.txt' (5 runs):

          1,586.40 msec task-clock         ( +-  0.06% )
     7,045,990,115      cycles             ( +-  0.06% )
    16,821,604,786      instructions       ( +-  0.00% )
```

A3. Run the same command with `gzip -1` and `gzip -9` (add the flag before `-c`). **Which counter explains the difference in time**: instructions or IPC?

A4. Record a profile of gzip. It has no symbols, so the report shows **addresses**, not function names:

```
$ taskset -c 1 perf record -g gzip -c task1/input.txt > /dev/null
$ perf report --stdio --no-children --sort sym -g none
    14.01%  [.] 0x00000000000044b4
     7.41%  [.] 0x00000000000044b1
     7.37%  [.] 0x00000000000044af
```

Why is there no name? Compare with the report of `heat_base` in Part B.

## Part B: The Heat Simulation

A metal plate is an $N \times N$ grid of `double`s, stored row by row. In each step every cell becomes the **average of its four neighbours**, and a few fixed cells get hotter. Three versions are given, with the same output. Read the code and find the differences marked `FIX`.

- `heat_base.cpp`: the starting point.
- `heat_lookup.cpp`: FIX 2. A table replaces a search.
- `heat_fixed.cpp`: FIX 2 and FIX 1. The loops are swapped.

```
$ make heat_base.out heat_lookup.out heat_fixed.out
$ taskset -c 1 perf stat -e task-clock,cycles,instructions,\
branch-misses,L1-dcache-loads,L1-dcache-load-misses ./heat_base.out
```

B1. Run `perf stat` on all three. **Which counter does each fix reduce?** Which program has the highest IPC, and is it the fastest?

B2. Run `perf record -g ./heat_base.out`, then `perf report --stdio --no-children`. **Which function takes most of the time?** Match it to a line of the code.

### What to Expect

The values depend on your machine. Compare the **changes**.

| | `heat_base` | `heat_lookup` | `heat_fixed` |
|---|---|---|---|
| time (ms) | 4055 | 659 | 355 |
| instructions | 51.6 G | 4.96 G | 4.95 G |
| L1 misses | 177 M | 219 M | 24 M |
| IPC | 2.69 | 1.55 | 2.89 |

gzip: IPC about 2.4, branch miss rate about 0.1%. `gzip -9` runs several times longer than `gzip -1`.

## Task 2: Branches and Memory

Two small programs, each with **one problem from Lab 8**. Find it with `perf`, fix it, and compare the counters before and after. Both are built with `-O0`, so the machine code follows your C++ line by line.

Each program runs a **naive** version (do not edit it), then **yours**, and prints `PASS` with the speedup if the results match. The argument `naive` or `yours` runs only that part, so you can use `perf` on it alone.

```
$ make repeated_array                    # naive, yours, PASS/FAIL
$ make repeated_array.out                # only build
$ taskset -c 1 perf stat -e ... ./repeated_array.out naive
```

### Part A: `repeated_array.cpp (TODO)`

An array `arr` of random `int`s in $[0, N)$ is processed many times:

```
for (int i = 0; i < REPS; i++)
    for (int j = 0; j < N; j++)
        if (arr[j] > N / 2) accum += i * arr[j];
        else                accum -= i * arr[j];
```

Write only in the marked spaces and keep the two loops. Please do not use algebra to skip work: the point is how the hardware runs this loop.

### Part B: `weird_matrix.cpp (TODO)`

Two $1000 \times 1000$ matrices `m1`, `m2`. For every pair `(i, j)` the program adds or subtracts a dot product of length 1000, for example:

```
for (int k = 0; k < N; k++)
    dotP += m1[arr[k]][i] * m2[k][arr[j]];
```

Write anything between `YOUR CODE BELOW` and `YOUR CODE ABOVE`. The matrices are `volatile`, so every read goes to memory; your copies need not be.

### Measuring

Follow **How to work on Tasks 2 to 5** and fill `task2/findings.txt` before you change code. Both programs spend their time in `main`, so **go one level deeper**: sample the event you suspect, then annotate `main` to see which source lines collect the samples.

```
$ taskset -c 1 perf record -e branch-misses ./repeated_array.out naive
$ perf annotate --stdio -l -M intel main
```

### Hints

- A: how many branch misses happen per loop iteration? Recall that in Lab 8 Task 3 we removed a branch by **turning the comparison into a number** (0 or 1) and using it in

arithmetic.

- B: in `m[r][c]`, which index changes in the innermost loop? Recall from Lab 8 Task 2 that this decides how far apart in memory two consecutive reads are. You can either reorder your loops, or copy the columns you need into rows first.

### Advanced Hints (outside the course)

- A: at `-O0` every local variable lives on the stack. Measure before you rely on a local accumulator.
- B: about 38% of the values in `arr` repeat, so some dot products are computed twice. Avoiding that changes the algorithm; do it after the cache fix.

### Testing

`make task2-check` checks that the code outside the marked spaces is unchanged and that both programs print `PASS`.

### What to Expect

The changes your fix should make (our machine, `taskset -c 1`):

| | `repeated_array` | | `weird_matrix` | |
|---|---|---|---|---|
| | naive | fixed | naive | fixed |
| time (ms) | 2659 | 1140 | 2565 | 1003 |
| branch-misses | 334 M | 0.4 M | 2.6 M | 2.5 M |
| L1 misses | 50 M | 50 M | 1848 M | 80 M |

A removes almost all branch misses and is about **2**× faster. B cuts L1 misses by more than **20**× and is about **2.5**× faster.

# Task 3: SIMD

**SIMD** instructions do the same arithmetic on **several numbers at once**. In Lab 8 Task 4 you used SSE with four `floats` per register. Here you use **SSE2 with `doubles`, two per register**. Everything is built with `-O0`, so the compiler does not vectorise anything for you.

### SSE2 with `doubles`

An `xmm` register holds **two `doubles`** (two lanes) and one instruction works on both. These intrinsics (header `<immintrin.h>`) are enough. The **Appendix: SIMD intrinsics** at the end lists more, including comparisons and masks:

| | |
|---|---|
| `__m128d` | type: 2 doubles |
| `_mm_loadu_pd(p)` / `_mm_storeu_pd(p, v)` | load / store `p[0]`, `p[1]` |
| `_mm_set1_pd(x)` / `_mm_setzero_pd()` | `x` in both lanes / zeros |
| `_mm_add_pd`, `_mm_sub_pd`, `_mm_mul_pd` | lane by lane $+$, $-$, $\times$ |
| `_mm_max_pd(a, b)` | lane by lane maximum |

This loop scales an array two elements at a time. When `n` is odd, a plain loop handles the **leftover** element:

```
__m128d s2 = _mm_set1_pd(s);
int j = 0;
for (; j + 2 <= n; j += 2) {
    __m128d v = _mm_loadu_pd(&a[j]);
    _mm_storeu_pd(&c[j], _mm_mul_pd(s2, v));
}
for (; j < n; j++)  c[j] = s * a[j];
```

A sum kept in a register has one partial sum per lane. Store it and add the two lanes at the end.

### Part A: `mask.cpp (TODO)`

A two-layer network with `NUM_CELL` $= 2048$ cells and `INPUT_DIM` $= 8192$ inputs. Cell $i$ has a weight vector $w_i$:

$$y_i = \sum_j \max(0,\ w_{i,j} \cdot x_j), \qquad \text{out} = \sum_{i=1}^{\texttt{NUM_CELL}-1} \max\big(y_i,\ (y_{i-1}+1) \cdot y_i\big).$$

Write the SIMD version between the markers; put the layer-1 outputs in `layer1OutsSIMD`. The check is exact: all values are whole numbers, so the order of additions does not matter.

### Part B: `optim.cpp (TODO)`

`naive.cpp` has plain versions of three functions on the `matrix` class. Write the same three in `optim.cpp` with SSE2. Keep the three signatures; everything else in the file is yours.

- `outer(a, b)`: outer product, $c[ir_2+p][jc_2+q] = a[i][j] \cdot b[p][q]$.
- `convolve(a, b)`: $b$ is a kernel with odd sides; $c[i][j]$ is the sum of $a[i+k][j+l]$ times the matching kernel value. The border, where the kernel does not fit, stays 0.
- `weirdMul(a, b)`: a matrix product in which each $a[i][k]$ is first replaced by $a[i][k]-1$ if it is $\leq 0.5$.

`./optim.out nocheck` skips the correctness check, so `perf stat` counts only the three functions. The sizes (63, 511, 401) are not multiples of 2 on purpose: **handle the leftover elements**. Do not edit `main.cpp`, `naive.cpp` or the headers.

### Measuring

Follow **How to work on Tasks 2 to 5** with `task3/findings.txt`. Measure the baseline with `./mask.out naive` and `./naive.out nocheck`; `make naive` and `make optim` print each function's time.

### Hints

- For each loop, ask which iterations are **independent and next to each other in memory**. Those become the two lanes.
- `outer`: for fixed $(i, j, p)$ a whole row of $b$ is multiplied by the same $a[i][j]$. Broadcast it with `_mm_set1_pd`.
- `convolve`: outputs $c[i][j]$ and $c[i][j+1]$ use the same kernel value and two neighbouring inputs.
- `weirdMul`: fix the loop order first (recall the `ikj` order from Lab 8 Task 2), then use SIMD on the inner loop.
- `mask`, layer 2: lane $i$ needs $y_{i-1}$. An unaligned load that starts one element earlier gives it.
- At `-O0` each `element()` call is a real function call. Take `double* row = m->element(i, 0);` once per row.

### Optional: Wider Registers

With AVX (4 doubles, `_mm256_`) or AVX-512 (8 doubles, `_mm512_`) build with `make optim SIMD=avx` or `SIMD=avx512`, and the same for `mask`. The lab machines have AVX but not AVX-512. Which functions gain from wider registers, and which do not?

### Testing

`make task3-check` checks that the fixed files are unchanged and that both programs print `PASS`. A wrong function gets `(wrong result)` after its time in `make optim`.

### What to Expect

Speedups over naive with SSE2, then AVX and AVX-512 (our machine):

| | naive (ms) | SSE2 | AVX | AVX-512 |
|---|---|---|---|---|
| `mask` | – | 6.5× | 11× | 14× |
| `outer` | 135 | 2.3× | 2.1× | 2.3× |
| `convolve` | 1252 | 2.3× | 4.2× | 6.8× |
| `weirdMul` | 357 | 3.6× | 6.9× | 8.1× |

`outer` writes 126 MB and does one multiplication per value written, so it waits on memory and wider registers do not help it.

## Task 4: Profiling a Program

Real programs have **several smaller problems in different functions**. This time you are not told where they are: `perf` will show you. The program is built with `-O2`, like real programs, so the compiler may **inline** small functions (copy them into their callers). An inlined function does not show up in `perf report` on its own.

### The Program

`grid_bfs.cpp (TODO)` builds a $260 \times 260$ city grid, finds 1200 shortest routes with breadth-first search (BFS) while counting visits per cell, then runs a congestion pass over the counts. Start reading at `main`:

```
main
 |- generate_grid, generate_requests
 |- run_all_requests -> shortest_path_bfs (x1200)
 |- summarize_heatmap
 |- compute_congestion_pressure -> next_pressure_value
 '- print_summary
```

Every printed line except `time_sec` must stay the same. Apart from that, you may change anything in the file.

### Measuring

Follow **How to work on Tasks 2 to 5** with `task4/findings.txt`:

```
$ make grid_bfs
$ taskset -c 1 perf stat -e task-clock,cycles,instructions,branch-misses,\
L1-dcache-loads,L1-dcache-load-misses,page-faults ./grid_bfs.out
$ taskset -c 1 perf record -g ./grid_bfs.out
$ perf report --stdio --no-children --sort sym -g none
```

Also run `time ./grid_bfs.out`. Part of the time is `sys`: the kernel handing the program fresh pages of memory (`page-faults`). **What in the code asks for so much fresh memory?**

The program also has a **memory leak**. Valgrind lists every allocation that was never freed; use the small run:

```
$ valgrind --leak-check=full ./grid_bfs.out --small
```

After your fix it prints `All heap blocks were freed`.

### Hints

- `compute_congestion_pressure`: the arrays are indexed `row * cols + col`. Recall Lab 8 Task 2: which loop is the inner one, and does it walk along a row?
- `next_pressure_value` is called 272 million times. What does a call cost compared with the arithmetic inside? A `static` function can be inlined.
- `shortest_path_bfs` runs 1200 times. What does it allocate on every call? Allocate once and reuse.

- Change one thing at a time and measure after each.

### Advanced Hints (outside the course)

- A single flat array for the grid instead of `vector<string>`.
- `distance` and `visited` can be one array of `uint16_t`.
- `(unsigned)r >= (unsigned)rows` checks `r < 0` and `r >= rows` at once.

### Testing

`make task4-check` runs the program and compares every line except `time_sec` with `task4/expected_output.txt`.

### What to Expect

The change each fix makes (our machine, `perf stat -r 3`). Times vary by about 10% from run to run; instructions, misses and page faults do not.

| | time (ms) | instructions | L1 misses | page faults |
|---|---|---|---|---|
| given | 1892 | 23.9 G | 1063 M | 99 600 |
| + loop order | 1820 | 23.1 G | 253 M | 99 596 |
| + `static` function | 1253 | 13.5 G | 252 M | 99 596 |
| + buffers allocated once | 1158 | 12.8 G | 231 M | 650 |

The loop order cuts L1 misses four times but the time only a little: the arrays are small (270 KB) and fit in L2. Each fix helps a different counter.

## Task 5: Profiling at Two Sizes

**Where a program spends its time depends on its input.** Profile one program with a small and a large table, fix what you find, and keep the answer the same. The program is built with `-O2`.

### The Program

`packets.cpp (TODO)` processes sensor packets in rounds. Each round (`run_epoch`) calls `refresh_history`, `process_packets` (which uses `branchy_score` and `chase_dependency`) and `cold_column_probe`. It prints one number. Run it with `history_cols` = 128 and = 2048:

```
$ make packets args=128
$ make packets args=2048
```

### Rules

- Do not edit the `build_` functions.
- Do not change any function's name, parameters or return type, and keep the existing calls. A call may pass a different variable of the same type.
- Keep `__attribute__((noinline))` on the hot functions; otherwise `-O2` inlines them and `perf report` shows only `main`.

### Measuring

Follow **How to work on Tasks 2 to 5** with `task5/findings.txt`. **Profile both sizes before changing anything and compare the two hotspot lists.** There is more than one problem and more than one way to fix each. At each step fix the one that gives the most for the least change, then measure again.

On AMD processors a `branch-misses` sample lands a little after the branch that missed. Treat sampled locations as hints and **confirm them with a change and `perf stat`**.

### Hints

- `cold_column_probe` walks down columns; recall Lab 8 Task 2. Does the order of the additions change the sum?
- `chase_dependency`: each load needs the previous one, as in Lab 8 Task 1. The 7-step walk from a start index is always the same. What can be computed once?
- `branchy_score`: the `if`s test bits that change randomly from packet to packet, like the branch in Lab 8 Task 3.

### Advanced Hints (outside the course)

- For a power of two, `x % n` equals `x & (n - 1)` when $x \geq 0$, and `&` is much cheaper.
- `int m = -cond;` is all ones or all zeros, so `(m & a) | (~m & b)` picks `a` or `b`.

### Testing

`make task5-check` checks the `build_` functions and compares the answer for both sizes with `task5/expected_output.txt`.

### What to Expect

The change each fix makes (our machine, `perf stat -r 5`):

| | 128 columns | | 2048 columns | |
|---|---|---|---|---|
| | ms | L1 misses | ms | L1 misses |
| given | 211 | 109 M | 2533 | 366 M |
| + `cold_column_probe` by rows | 199 | 94 M | 482 | 127 M |
| + 7-step totals computed once | 112 | 21 M | 373 | 53 M |
| + branchless `branchy_score` | 73 | 21 M | 327 | 53 M |

At 128 columns the history table is 2 MB; at 2048 it is 32 MB. Which function would you expect at the top for 512 columns? Run it and check.

# Task 6 (optional): Optimising a Program

**This task is optional and not graded. You are not expected to complete it.** It is a substantial C++ problem. It shows how far the ideas of Tasks 2 to 5 can take one program.

## The Game

A square grid of $N \times N$ **cells**, where each cell is `EMPTY` (0), `EGG` (1), `JUVENILE` (2) or `ADULT` (3). The **neighbourhood** of a cell is the $5 \times 5$ **block around it**, without the cell itself: 24 cells. **The grid wraps around**: the left edge touches the right edge, and the top touches the bottom. Let $A$ be the number of `ADULT`s in the neighbourhood. **All cells change at the same time**:

| now | next generation |
|---|---|
| `EMPTY` | `EGG` if $3 \leq A \leq 5$, else `EMPTY` |
| `EGG` | `JUVENILE` |
| `JUVENILE` | `ADULT` |
| `ADULT` | stays `ADULT` if $4 \leq A \leq 9$, else `EMPTY` |

`spawn_ref.cpp` computes this directly: for every cell, 24 reads with `%` for the wrap-around, then a `switch`. Do not edit it; it produces the expected outputs.

## What You Need to Do

`spawn_sim.cpp (optional)` starts as a copy of the reference. **Make it fast. The output file must stay byte for byte identical.**

**You may not use an algorithm that does less work**, such as remembering earlier grids or skipping generations: every generation must be computed. The speed has to come from **memory layout, SIMD, threads and cache behaviour**.

```
$ make task6-grids     # once: 10 input grids (512 and 1024) and their outputs
                       # after 1000 generations from spawn_ref (about a minute;
                       # needs python3 with numpy)
$ make task6           # build spawn_sim.cpp (-O3 -march=native), run all grids,
                       # compare with the expected output, print the speedup
```

## A Path Through It

These are the steps that worked for us, roughly in order. Each one is a separate exercise; stop wherever you like.

1. **Count adults once, not 24 times.** Neighbouring cells share most of their $5 \times 5$ blocks. Sum each row of 5 once (a horizontal sum), then add 5 of those row sums vertically. Moving down one row, the vertical sum changes by one row in and one row out: a *sliding window*. This $5 \times 5$ sum includes the cell itself, so subtract 1 when the cell is an `ADULT`, or shift the rule thresholds by one.
2. **Remove the %.** Copy 2 extra rows and columns around the grid (a *halo*), so the inner loops never wrap.
3. **SIMD.** One cell is one byte, so an SSE2 register holds 16 cells and an AVX2 register 32. Compare 16 cells with 3 at once (`_mm_cmpeq_epi8` gives `0xFF` or `0` per byte), add the 0/1 results, and apply the rules with compares and masks instead of a `switch` (Task 3, Lab 8 Task 4). The byte intrinsics are in the **Appendix: SIMD intrinsics**.

4. **Threads.** Give each thread a band of rows (`std::thread`). All threads must finish a generation before any starts the next (`std::barrier`). Pin each thread to a core.

5. **Bits instead of bytes** (hard). Store the grid as three bit-planes (`adult`, `juvenile`, `egg`), 64 cells per `uint64_t`. Counting then becomes adding bits with *carry-save adders* built from `&`, `|` and `^`: 64 cells per instruction, and 3 bits per cell instead of 8.

### Testing

`make task6` prints one line per grid. The copy you start with is exactly as fast as the reference:

```
$ make task6
PASS -- public_4_sparse_clusters_512.bin   2807.963 ms  (reference 2790.700 ms, 1.0x)
...
```

### Reference Values

$1024 \times 1024$, 1000 generations, machine A:

| version | time |
|---|---|
| `spawn_ref` | 11.6 s |
| bit-planes, no SIMD, one core | 6.5 s |
| AVX2 sliding window, 1 thread | 91 ms |
| the same with 8 threads | 30 ms (about 390×) |

## Appendix: `perf` reference

Table 1: `perf` commands used in this outlab

| Command | Does |
|---|---|
| `perf list` | list the events this CPU can count |
| `perf stat -e EV1,EV2 ./prog` | count events over the whole run |
| `perf stat -r 5 -e EV ./prog` | run 5 times, print the mean and spread |
| `perf stat -I 250 -e EV ./prog` | print the counts every 250 ms |
| `perf record -g ./prog` | sample where time goes, with call stacks |
| `perf record -e EV -c 10000 ./prog` | one sample per 10 000 events `EV` |
| `perf report --stdio --no-children --sort sym -g none` | functions by self share |
| `perf report --stdio --children --sort sym -g none` | functions by self + callees |
| `perf report --stdio` | the same with call chains |
| `perf annotate --stdio -l -M intel 'f'` | code of `f` with samples per line |

Put `taskset -c 1` in front of `perf stat` and `perf record` to pin the run. `perf record` writes `perf.data` in the current folder; `perf report` and `perf annotate` read it. For `f`, copy the function name exactly as `perf report` prints it.

Table 2: `perf` events used in this outlab

| Event | Counts |
|---|---|
| `task-clock` | milliseconds the program ran on the CPU |
| `cycles` | clock cycles of the core |
| `instructions` | instructions that finished |
| `branches` | branch instructions executed |
| `branch-misses` | branches that were predicted wrong |
| `L1-dcache-loads` | reads from the L1 data cache |
| `L1-dcache-load-misses` | reads that were not in L1 |
| `page-faults` | times the kernel mapped a fresh page for the program |
| `context-switches` | times the program was taken off the core |
| `cpu-migrations` | times it moved to another core |

IPC = `instructions / cycles`. A percentage in brackets at the end of a `perf stat` line, such as `(83.33%)`, means the CPU had too few counters for all events at once: `perf` counted that event for that share of the time and scaled the count up.

# Appendix: SIMD intrinsics

Header: `<immintrin.h>`. Each row gives the SSE2 name (2 `double`s, type `__m128d`). **AVX** (4 doubles, `__m256d`) and **AVX-512** (8 doubles, `__m512d`) use the same name with `_mm256_` or `_mm512_` instead of `_mm_`. AVX and AVX-512 need `make ... SIMD=avx` or `SIMD=avx512`.

Table 3: Intrinsics on `double`s (Task 3)

| Intrinsic | Does |
|---|---|
| `_mm_loadu_pd(p)` | load `p[0]`, `p[1]` into a register (any alignment) |
| `_mm_storeu_pd(p, v)` | store the lanes of `v` to `p[0]`, `p[1]` |
| `_mm_set1_pd(x)` | `x` in every lane |
| `_mm_setzero_pd()` | every lane 0 |
| `_mm_add_pd(a, b)`, `_mm_sub_pd(a, b)` | lane by lane $a+b$, $a-b$ |
| `_mm_mul_pd(a, b)`, `_mm_div_pd(a, b)` | lane by lane $a \cdot b$, $a/b$ |
| `_mm_max_pd(a, b)`, `_mm_min_pd(a, b)` | lane by lane maximum, minimum |
| `_mm_cmpgt_pd(a, b)` | per lane: all ones if $a > b$, else all zeros (a *mask*) |
| `_mm_and_pd(a, b)`, `_mm_or_pd(a, b)` | bitwise and, or |
| `_mm_andnot_pd(a, b)` | bitwise $(\sim a)$ and $b$ |
| `_mm_cvtsd_f64(v)` | lane 0 as a plain `double` |

A **select** without a branch: with `m = _mm_cmpgt_pd(a, b)`, the lanes of `_mm_or_pd(_mm_and_pd(m, x), _mm_andnot_pd(m, y))` are `x` where $a > b$ and `y` elsewhere. To add the lanes of a register, store it to a `double[2]` and add the two elements.

Table 4: Intrinsics on bytes (Task 6, optional): 16 cells per SSE2 register, `__m128i`

| Intrinsic | Does |
|---|---|
| `_mm_loadu_si128((const __m128i*)p)` | load 16 bytes |
| `_mm_storeu_si128((__m128i*)p, v)` | store 16 bytes |
| `_mm_set1_epi8(x)` | `x` in all 16 bytes |
| `_mm_setzero_si128()` | all bytes 0 |
| `_mm_add_epi8(a, b)`, `_mm_sub_epi8(a, b)` | byte by byte $a+b$, $a-b$ |
| `_mm_cmpeq_epi8(a, b)` | per byte: `0xFF` if equal, else `0` |
| `_mm_cmpgt_epi8(a, b)`, `_mm_cmplt_epi8(a, b)` | per byte: `0xFF` if $a > b$ (signed), $a < b$ |
| `_mm_and_si128(a, b)`, `_mm_or_si128(a, b)` | bitwise and, or |
| `_mm_andnot_si128(a, b)` | bitwise $(\sim a)$ and $b$ |

The compare result `0xFF` is $-1$ as a signed byte, so *subtracting* a compare result from a counter adds 1 per true lane. The AVX2 versions (32 bytes) use `_mm256_` and `_si256`.